# TALLER APIGEE

SESION 1

# OBJETIVOS

## TEORICO

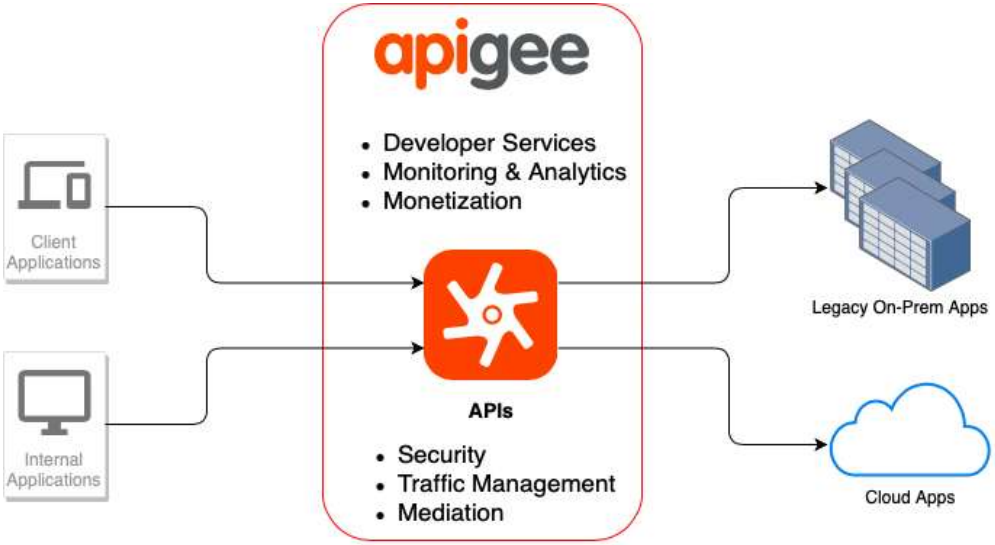
1. ¿Qué es la integración de Servicios?
2. ¿Qué es una API?
3. ¿Qué significa API?
4. API REST
5. JSON, XML, YAML
6. API MANAGEMENT APIGEE

## PRACTICO

1. API PROXY
2. SHARED FLOW
3. ENVIROMENT CONFIGURATION
4. PRODUCT
5. DEVELOP
6. APP
7. HOLA MUNDO

# ¿QUÉ ES LA INTEGRACIÓN DE SERVICIOS?

- La integración de servicios es un conjunto de tecnologías que proporciona servicios de mensajería asíncrona, configurando un medio por el cual dos sistemas se comunican.




https://www.ibm.com/docs/es/was-nd/8.5.5?topic=technologies-service-integration

# ¿QUÉ ES UNA API?

- Las API son mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos. Por ejemplo, el sistema de software del instituto de meteorología contiene datos meteorológicos diarios. La aplicación meteorológica de su teléfono "habla" con este sistema a través de las API y le muestra las actualizaciones meteorológicas diarias en su teléfono.

https://aws.amazon.com/es/what-is/api/

# ¿QUÉ SIGNIFICA API?

- API significa "interfaz de programación de aplicaciones". En el contexto de las API, la palabra aplicación se refiere a cualquier software con una función distinta. La interfaz puede considerarse como un contrato de servicio entre dos aplicaciones. Este contrato define cómo se comunican entre sí mediante solicitudes y respuestas. La documentación de su API contiene información sobre cómo los desarrolladores deben estructurar esas solicitudes y respuestas.

https://aws.amazon.com/es/what-is/api/

# API REST

## ¿Cómo Funcionan las API?

- La arquitectura de las API suele explicarse en términos de cliente y servidor. La aplicación que envía la solicitud se llama cliente, y la que envía la respuesta se llama servidor. En el ejemplo del tiempo, la base de datos meteorológicos del instituto es el servidor y la aplicación móvil es el cliente.

## API de REST

- Estas son las API más populares y flexibles que se encuentran en la web actualmente. El cliente envía las solicitudes al servidor como datos. El servidor utiliza esta entrada del cliente para iniciar funciones internas y devuelve los datos de salida al cliente.

https://aws.amazon.com/es/what-is/api/

# API REST

- REST significa transferencia de estado representacional. REST define un conjunto de funciones como GET, PUT, DELETE, etc. que los clientes pueden utilizar para acceder a los datos del servidor. Los clientes y los servidores intercambian datos mediante HTTP.
- La principal característica de la API de REST es que no tiene estado. La ausencia de estado significa que los servidores no guardan los datos del cliente entre las solicitudes. Las solicitudes de los clientes al servidor son similares a las URL que se escriben en el navegador para visitar un sitio web. La respuesta del servidor son datos simples, sin la típica representación gráfica de una página web.

https://blog.thedojo.mx/2019/08/03/entendiendo-rest-servidor-sin-estado.html

https://aws.amazon.com/es/what-is/api/

# ESTRUCTURAS DE DATOS

## XML

- El lenguaje de marcado extensible (XML) permite definir y almacenar datos de forma compartible. XML admite el intercambio de información entre sistemas de computación, como sitios web, bases de datos y aplicaciones de terceros. Las reglas predefinidas facilitan la transmisión de datos como archivos XML a través de cualquier red, ya que el destinatario puede usar esas reglas para leer los datos de forma precisa y eficiente.

XML

```
<apis>
    <api>
        <name>login</name>
        <port>8080</port>
    </api>
    <api>
        <name>profile</name>
        <port>8090</port>
    </api>
</apis>
```

https://aws.amazon.com/es/what-is/xml/

# ESTRUCTURAS DE DATOS

## JSON

- JSON, o JavaScript Object Notation, es un formato de intercambio de datos ligero que se utiliza habitualmente para intercambiar datos entre aplicaciones web.

JSON

```
{
  "apis": [
    {
      "name": "login",
      "port": 8080
    },
    {
      "name": "profile",
      "port": 8090
    }
  ]
}
```

https://www.dongee.com/tutoriales/que-es-json/

# ESTRUCTURAS DE DATOS

## YAML

- YAML es un formato de serialización de datos cuyas siglas significan **YAML ain't Markup language**
- La principal ventaja de utilizar YAML es su legibilidad y capacidad de escritura. Si tiene un archivo de configuración que debe ser más fácil de leer para los humanos, es mejor que utilice YAML. YAML no es una sustitución completa de JSON ya que JSON y XML también tienen su lugar; sin embargo, es útil aprender YAML

https://geekflare.com/es/yaml-introduction/

YAML

```yaml
apis:
  - name: login
    port: 8080
  - name: profile
    port: 8090
```

# API MANAGEMENT APIGEE

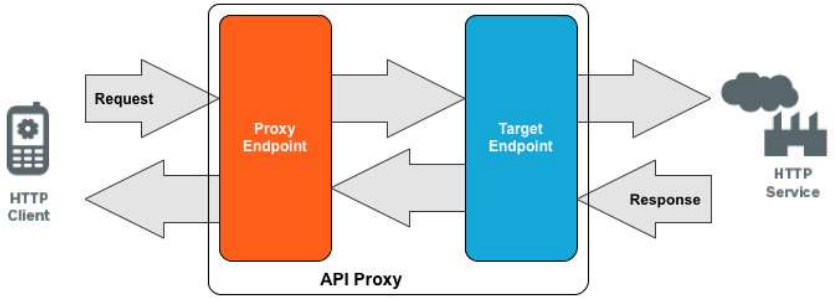
## ¿Qué es apigee?

- Apigee es una plataforma para desarrollar y administrar la API. Mediante el fronting de servicios con una capa de proxy, Apigee proporciona una abstracción o fachada para tus API de servicio de backend y proporciona seguridad, límite de frecuencia, cuotas, estadísticas y mucho más.

https://cloud.google.com/apigee/docs/api-platform/get-started/what-apigee?hl=es-419

# API PROXY

- Para exponer las API en Apigee Edge, debes implementar *proxies de API*. Los proxies de API separan la API orientada a la aplicación de sus servicios de backend, lo que protege a esas aplicaciones de los cambios en el código de backend. Aunque haga cambios de backend en sus servicios, las aplicaciones seguirán llamando a la misma API sin ninguna interrupción.



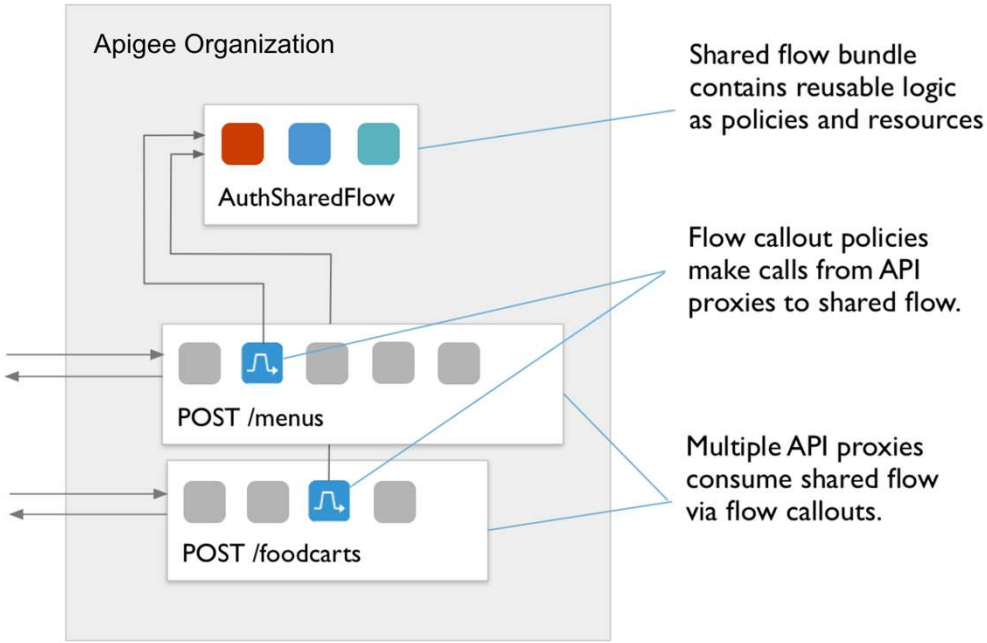

```
<ProxyEndpoint name="default">
    <PreFlow/>
    <Flows/>
    <PostFlow/>
    <HTTPProxyConnection>
        <VirtualHost>default</VirtualHost>
        <BasePath>/v1/weather</BasePath>
    </HTTPProxyConnection>
    <RouteRule name="default">
        <TargetEndpoint>default</TargetEndpoint>
    </RouteRule>
</ProxyEndpoint>
```

```
<TargetEndpoint name="default">
    <PreFlow/>
    <Flows/>
    <PostFlow/>
    <HTTPTargetConnection>
        <URL>http://weather.yahooapis.com</URL>
    </HTTPTargetConnection>
</TargetEndpoint>
```

https://cloud.google.com/apigee/docs/api-platform/fundamentals/understanding-apis-and-api-proxies?hl=es-419

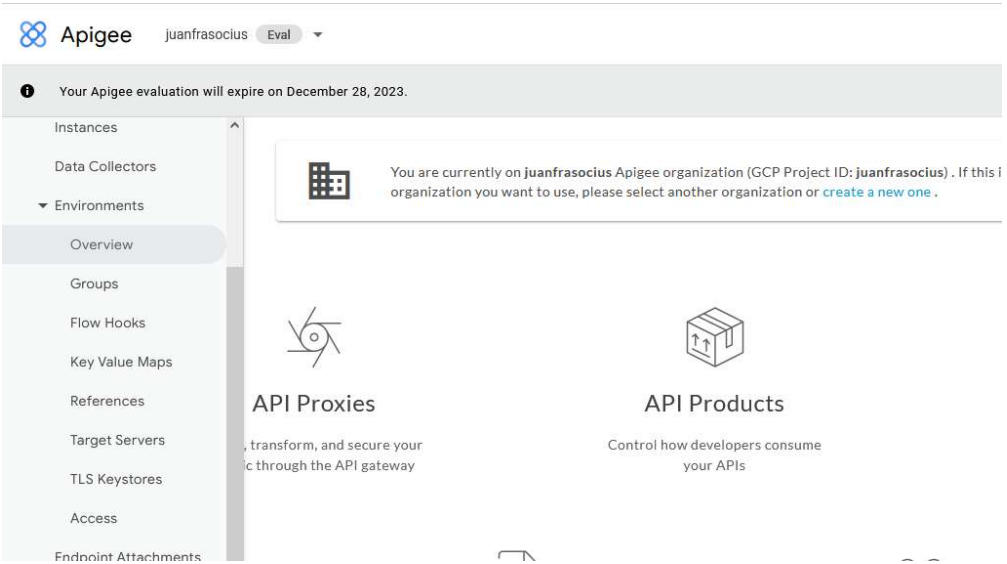
# SHARED FLOW

- Combina políticas y recursos en un *flujo compartido* que puedes consumir desde varios proxies de API, y también desde otros flujos de uso compartido. Aunque es como un proxy, un flujo compartido no tiene extremo. Solo se puede usar desde un proxy de API o un flujo compartido que se encuentre en la misma organización que el flujo compartido en sí.




https://cloud.google.com/apigee/docs/api-platform/fundamentals/shared-flows?hl=es-419

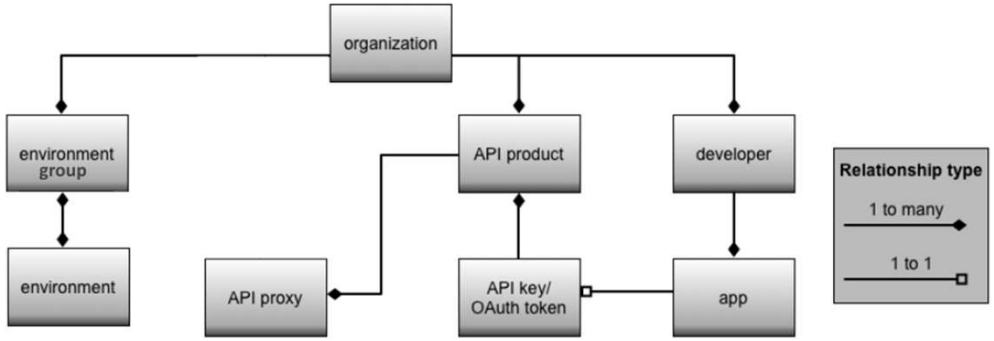
# ENVIROMENT CONFIGURATION




- Permiten configurar ambientes, grupos de servidores, Certificados, Target, etc.

# PRODUCT

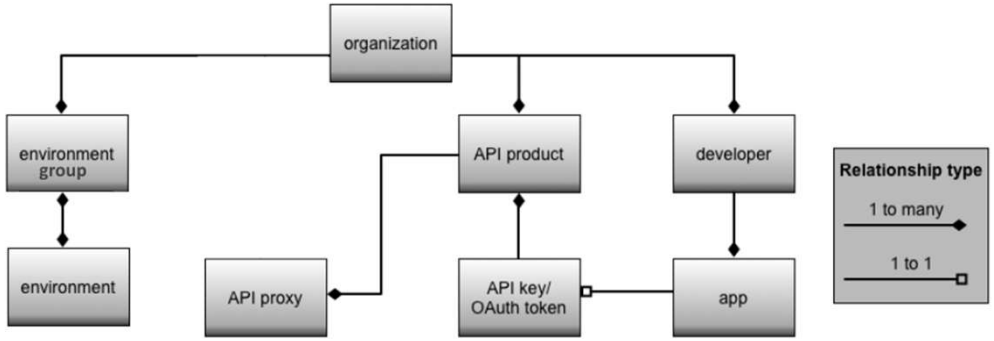
- Un paquete de proxies de API combinado con un plan de servicio. Ese plan de servicio puede especificar límites de acceso en los proxies de API, proporcionar seguridad, permitir la supervisión y las estadísticas y proporcionar funciones adicionales.




https://cloud.google.com/apigee/docs/api-platform/fundamentals/organization-structure?hl=es-419

# DEVELOP

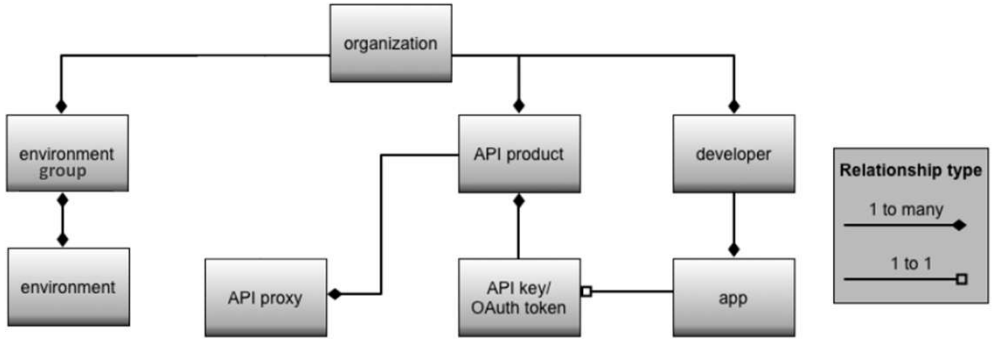
- Una organización contiene uno o más desarrolladores que compilan las apps que consumen las API (ensambladas en productos de API) definidas por la organización. Los desarrolladores consumen API, pero no pueden crear API ni realizar ninguna otra acción en la organización.




https://cloud.google.com/apigee/docs/api-platform/fundamentals/organization-structure?hl=es-419

# APP

- Los desarrolladores deben registrar sus apps en tu organización. Una app es una representación de la app real del desarrollador que le proporciona una clave de API para pasar cada solicitud a las API.




https://cloud.google.com/apigee/docs/api-platform/fundamentals/organization-structure?hl=es-419

# HOLA MUNDO

# PREGUNTAS

JUAN.RAMIREZ@SOCIUSCHILE.CL